



Отчет о проверке

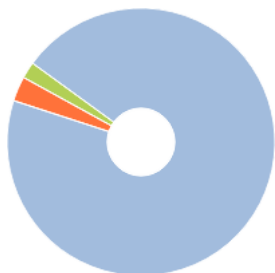
Автор: Багаев Аркадий Асланович

Название документа: Kurs_rabota.docx

Проверяющий: Эксперт СОГУ

Организация: ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ "СЕВЕРО-ОСЕТИНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ КОСТА ЛЕВАНОВИЧА ХЕТАГУРОВА"

РЕЗУЛЬТАТЫ ПРОВЕРКИ



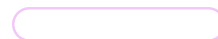
Совпадения:
3,09%



Оригинальность:
95,26%



ИИ-контент:
0%



Цитирования:
1,65%



Самоцитирования:
0%



«Совпадения», «Цитирования», «Самоцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.



Проверено: 76,39% текста документа, исключено из проверки: 23,61% текста документа. Разделы и элементы, отключенные пользователем: Библиография, Таблицы

- Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самоцитированию. Показатель «Совпадения» — это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- Самоцитирования** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самоцитирования» — это доля фрагментов текста, отнесенных к самоцитированию, в общем объеме текста.
- Цитирования** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы; библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» — это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» — это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 22907

Тип документа: Не указано

Дата проверки: 17.06.2026 00:23:26

Дата корректировки: Нет

Количество страниц: 24

Символов в тексте: 24325

Слов в тексте: 2755

Число предложений: 1657

Комментарий: не указано

ПАРАМЕТРЫ ПРОВЕРКИ

Выполнена проверка с учетом редактирования: Да

Выполнено распознавание текста (OCR): Нет

Выполнена проверка с учетом структуры: Да

Разделы и элементы, отключенные пользователем: Библиография, Таблицы

Модули поиска: Перефразирования по коллекции IEEE, Профессиональная лексика. Медицина, Цитирование, Перефразирования по СПС ГАРАНТ: аналитика, Профессиональная лексика. АПК и биотех, Переводные заимствования, Патенты СССР, РФ, СНГ, PubMed, Переводные заимствования по Коллекции открытых публикаций международных издательств, Перефразирования по Коллекции открытых публикаций международных издательств, Шаблонные фразы, Коллекция открытых публикаций международных издательств, Профессиональная лексика. Юриспруденция, IEEE, Перефразирования по базе публикаций открытого доступа PubMed, Переводные заимствования IEEE, СМИ России и СНГ, ИПС Адилет, Кольцо вузов, Публикации eLIBRARY, Кольцо вузов (переводы и перефразирования), Публикации РГБ, Публикации РГБ (переводы и перефразирования), Переводные заимствования по коллекции Интернет в русском сегменте, СПС ГАРАНТ: аналитика, Публикации eLIBRARY (переводы и перефразирования), Перефразированные заимствования по коллекции Интернет в английском сегменте, Перефразированные заимствования по коллекции Интернет в русском сегменте, Коллекция НБУ, Переводные заимствования по коллекции Гарант: аналитика, Переводные заимствования по базе публикаций открытого доступа PubMed, СПС ГАРАНТ: нормативно-правовая документация, Сводная коллекция научных работ Беларуси, Медицина, Переводные заимствования по коллекции Интернет в английском сегменте, Сводная коллекция ЭБС, Интернет Плюс, Собственная коллекция компании, Собственная коллекция (переводы и перефразирования)

ИСТОЧНИКИ

№	Доля в тексте	Доля в отчете	Источник	Актуален на	Модуль поиска	Комментарий
[01]	2,37%	0,74%	Сайт для изучения английского я...	11 Янв 2024	Собственная коллекция компании	
[02]	2,37%	0%	Русско-осетинский словарь в ма...	11 Янв 2024	Собственная коллекция компании	
[03]	2,37%	0%	Личный кабинет и регистрация ...	11 Янв 2024	Собственная коллекция компании	
[04]	2,33%	0%	Теория диплома	10 Янв 2024	Собственная коллекция компании	
[05]	1,65%	1,65%	не указано	13 Янв 2022	Шаблонные фразы	
[06]	1,62%	0%	Языковая игра(осетинский, армя...	11 Янв 2024	Собственная коллекция компании	
[07]	1,44%	0,55%	RPV_NAPR_01_04_02_PMI_MMIVM_... https://dgu.ru	05 Июн 2025	Интернет Плюс	
[08]	1,35%	0%	туристический клуб (1)	11 Янв 2024	Собственная коллекция компании	
[09]	1,35%	0%	Пинегин	10 Янв 2024	Собственная коллекция компании	
[10]	1,32%	0%	https://dspace.tltsu.ru/bitstream/1... https://dspace.tltsu.ru	29 Янв 2024	Интернет Плюс	
[11]	1,16%	0,11%	Оптимизационные методы в нек...	15 Янв 2024	Кольцо вузов	
[12]	1,16%	0%	rp-vospitanija-i-kalend.plan_pmii-2... https://nosu.ru	23 Сен 2025	Интернет Плюс	
[13]	1,07%	1,07%	Подписание отчетности с исполь... http://ivo.garant.ru	03 Июн 2023	Переводные заимствования по коллекции Гарант: аналитика	
[14]	1,03%	0,62%	Курсовая работа Зарипова Екат...	22 Мая 2026	Кольцо вузов	
[15]	1%	0%	Совершенствование рекламной ...	31 Дек 2026	Собственная коллекция (переводы и перефразирования)	
[16]	1%	0%	Тотиков Сармат. ВКР	31 Дек 2026	Собственная коллекция (переводы и перефразирования)	
[17]	1%	0%	Багаев. ВКР	31 Дек 2026	Собственная коллекция (переводы и перефразирования)	
[18]	1%	0%	Мусофинов Мухаммад. ВКР	31 Дек 2026	Собственная коллекция (переводы и перефразирования)	

[19]	1%	0%	Повышение эффективности орга...	31 Дек 2026	Собственная коллекция (переводы и перефразирования)	
[20]	0,95%	0%	Методические рекомендации к н... http://elibrary.ru	01 Янв 2021	Публикации eLIBRARY	Источник исключен. Причина: Маленький процент пересечения.
[21]	0,95%	0%	ПОДГОТОВКА И ПРОВЕДЕНИЕ ИТ... http://elibrary.ru	01 Янв 2017	Публикации eLIBRARY	Источник исключен. Причина: Маленький процент пересечения.
[22]	0,95%	0%	Политика обеспечения ликвидно... http://biblioclub.ru	21 Янв 2020	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.
[23]	0,95%	0%	Немецкие слова в современном ... http://biblioclub.ru	21 Янв 2020	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.
[24]	0,89%	0%	Способ формирования водохрани... http://findpatent.ru	24 Июн 2015	Патенты СССР, РФ, СНГ	Источник исключен. Причина: Маленький процент пересечения.
[25]	0,89%	0%	Минеральные источники Бугульт... https://fips.ru	02 Апр 2026	Патенты СССР, РФ, СНГ	Источник исключен. Причина: Маленький процент пересечения.
[26]	0,89%	0%	Способ предотвращения загрязн... http://findpatent.ru	24 Июн 2015	Патенты СССР, РФ, СНГ	Источник исключен. Причина: Маленький процент пересечения.
[27]	0,89%	0%	Особенности приема на обучени... https://vladikavkaz.bezformata.com	05 Июл 2020	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[28]	0,89%	0%	Школа анализа данных. http://elibrary.ru	01 Янв 2022	Публикации eLIBRARY	Источник исключен. Причина: Маленький процент пересечения.
[29]	0,89%	0%	не указано http://ivo.garant.ru	02 Авг 2022	СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[30]	0,89%	0%	Приказ Министерства образован... http://ivo.garant.ru	14 Апр 2024	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[31]	0,89%	0%	Приказ Министерства образован... http://ivo.garant.ru	29 Апр 2025	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[32]	0,89%	0%	Приказ Министерства культуры Р... http://ivo.garant.ru	06 Июл 2025	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[33]	0,89%	0%	Приказ Министерства образован... http://ivo.garant.ru	29 Апр 2025	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[34]	0,82%	0%	27.06.2023 Гуманизм социологич...	11 Янв 2024	Собственная коллекция компании	Источник исключен. Причина: Маленький процент пересечения.
[35]	0,74%	0%	не указано https://openalex.org	28 Мар 2024	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[36]	0,74%	0%	Study of the spread of electron vel... https://ieeexplore.ieee.org	04 Ноя 2013	IEEE	Источник исключен. Причина: Маленький процент пересечения.
[37]	0,74%	0%	Костяно, Николай Олегович Фар... http://dlib.rsl.ru	01 Янв 2023	Публикации РГБ	Источник исключен. Причина: Маленький процент пересечения.
[38]	0,74%	0%	Коронавирус COVID-19 http://ivo.garant.ru	08 Фев 2020	СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[39]	0,74%	0%	Дефицит нейродинамических ко... http://emll.ru	21 Дек 2016	Медицина	Источник исключен. Причина: Маленький процент пересечения.
[40]	0,7%	0%	РОССИЙСКИЙ БИЗНЕС И КСО: ОС... https://openalex.org	21 Окт 2021	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[41]	0,7%	0%	Экономика и менеджмент: совре... http://elibrary.ru	01 Янв 2017	Публикации eLIBRARY	Источник исключен. Причина: Маленький процент пересечения.
[42]	0,7%	0%	Фокин, Иван Евгеньевич Разрабо... http://dlib.rsl.ru	01 Янв 2022	Публикации РГБ	Источник исключен. Причина: Маленький процент пересечения.
[43]	0,7%	0%	Отчет о результатах контрольног... http://ivo.garant.ru	06 Июл 2019	СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[44]	0,7%	0%	Постановление Правительства К... http://ivo.garant.ru	25 Дек 2017	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[45]	0,7%	0%	ВКР Автушкевич	11 Янв 2024	Собственная коллекция компании	Источник исключен. Причина: Маленький процент пересечения.
[46]	0,58%	0%	Полоролевая дифференциация б... https://elib.nlb.by	раньше 2011	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[47]	0,53%	0%	voronova_a_v_vyuvavlenie-akademic...	17 Июн 2021	Кольцо вузов	Источник исключен. Причина: Маленький процент пересечения.
[48]	0,52%	0%	Учет, анализ и аудит основных с... http://biblioclub.ru	21 Янв 2020	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.
[49]	0,48%	0%	Методы и технические средства ... http://diss.natlib.uz	02 Сен 2014	Коллекция НБУ	Источник исключен. Причина: Маленький процент пересечения.

[50]	0,44%	0%	Асимптотика и численное решен... http://diss.natlib.uz	02 Сен 2014	Коллекция НБУ	Источник исключен. Причина: Маленький процент пересечения.
[51]	0,44%	0%	Исследованиеи разработка мате... http://diss.natlib.uz	29 Авг 2014	Коллекция НБУ	Источник исключен. Причина: Маленький процент пересечения.
[52]	0,44%	0%	Математическое моделирование... http://diss.natlib.uz	13 Мар 2017	Коллекция НБУ	Источник исключен. Причина: Маленький процент пересечения.
[53]	0,44%	0%	Планирование себестоимости и ... https://elib.nlb.by	раньше 2011	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[54]	0,44%	0%	Совершенствование учета и ана... https://elib.nlb.by	раньше 2011	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[55]	0,44%	0%	Практикум по психологии профе... https://e.lanbook.com	21 Янв 2020	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.

Федеральное государственное бюджетное образовательное учреждение высшего образования «**5**еверо-Осетинский **11**дарственный университет имени Коста Левановича Хетагурова» **5**

Факультет математики и компьютерных наук
Кафедра прикладной математики и информатики

КУРСОВОЙ ПРОЕКТ **7**

на тему:

Разработка приложения-помощника для управления личными финансами и планирования задач

выполнен в рамках практики «Учебная технологическая
(проектно-технологическая) практика»

ВЫПОЛНИЛИ:

студент 2 курса ОФО
направления «01.03.02 Прикладная
математика и информатика»
Багаев А.А.

НАУЧНЫЙ РУКОВОДИТЕЛЬ:

Старший преподаватель
Кафедра **5**икладной математики и
информатики **1**
Воронцова И.А.

РУКОВОДИТЕЛЬ ПРАКТИКИ

ассистент
Бирагова Л.К.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
1. АНАЛИЗ И ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	6
1.1. Постановка задачи и обоснование актуальности.....	6
1.2. Функциональные и нефункциональные требования.....	6
1.3. Выбор средств разработки.....	7
1.4. Проектирование объектной модели.....	8
Выводы по 1 главе	9
2. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ.....	10
2.1. Структура проекта и описание классов.....	11
2.2. Реализация бизнес-логики (менеджер данных).....	11
2.3. Разработка графического интерфейса пользователя	12
2.4. Механизм сохранения и загрузки данных.....	14
Выводы по 2 главе.....	15
3. ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ.....	16
3.1. Сценарии тестирования и их результаты.....	16
3.2. Руководство пользователя.....	17
3.3. Примеры работы с приложением	18
Выводы по 3 главе.....	20
ЗАКЛЮЧЕНИЕ.....	20
СПИСОК ЛИТЕРАТУРЫ.....	21
4. ЛИПСИНГ ПРОГРАММНОГО КОДА.....	22

ВВЕДЕНИЕ

Актуальность темы.

В современном мире человеку необходимо одновременно контролировать личные финансы и эффективно планировать время. Согласно исследованиям, ведение учёта доходов и расходов позволяет экономить до 20% бюджета, а систематическое планирование задач повышает продуктивность на 30–40%. Существующие программные решения (Monefy, Trello, Todoist) часто либо избыточны, либо требуют платной подписки, либо не работают без интернета. Поэтому разработка простого, автономного десктопного приложения, объединяющего функции финансового трекера и планировщика задач, является актуальной.

Объект исследования – процессы управления личными финансами и планирования повседневных задач.

Предмет исследования – методы программной реализации системы учёта доходов/расходов и менеджера задач в среде Windows.

Цель работы – разработать на языке C# приложение-помощник, позволяющее пользователю вести учёт финансовых операций, отслеживать текущий баланс и управлять списком задач с возможностью указания сроков и приоритетов.

Гипотеза исследования – предполагается, что создание локального десктопного приложения с интуитивно понятным интерфейсом и встроенным механизмом автокассов позволит пользователю более дисциплинированно относиться к бюджету и задачам, а также снизит порог входа по сравнению со сложными онлайн-сервисами.

Задачи исследования:

1. Провести анализ предметной области и сформулировать требования к приложению.
2. Выбрать оптимальные средства разработки (язык, платформу, библиотеки).
3. Спроектировать и реализовать объектную модель (классы транзакций, задач, менеджера данных).

4. Разработать графический интерфейс пользователя на Windows Forms.
5. Реализовать механизм сохранения и загрузки данных в файлы без использования внешних библиотек.
6. Провести тестирование основных функций и составить руководство пользователя.

Методологическая база исследования.

Применены методы объектно-ориентированного анализа и проектирования, структурного программирования, тестирования «чёрного ящика». Использованы подходы: модульное программирование, разделение логики и представления, работа с потоками ввода-вывода.

Обзор источников и литературы.

Разработке приложений для управления финансами и задачами посвящены работы многих авторов. В учебнике Троелсена и Джепикса [1] детально рассмотрены возможности C# и платформы .NET для создания десктопных приложений. Шилдт [3] акцентирует внимание на объектно-ориентированном подходе, который лёг в основу проектирования классов Transaction и TaskItem. Вопросы работы с файловым вводом-выводом, использованные в механизме сохранения данных, освещены у Нагановского [5]. Документация Microsoft [4] содержит рекомендации по проектированию интерфейса Windows Forms. Анализ существующих аналогов (Monefy, Trello) показал, что они либо не имеют десктопной версии, либо избыточны, поэтому создание локального приложения-помощника остаётся актуальным.

Научная новизна заключается в создании целостного, автономного приложения, не требующего установки дополнительных библиотек, с оригинальным текстовым форматом хранения данных.

Практическая значимость – полученное приложение может использоваться любым пользователем Windows для повседневного контроля бюджета и планирования дел.

Структура работы.

Курсовая работа состоит из введения, трёх глав, заключения, списка литературы приложения. В первой главе проводится анализ требований и проектирование. Во второй главе описывается реализация 14 компонентов. Третья глава посвящена тестированию и руководству пользователя.

1. АНАЛИЗ И ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

1.1. Постановка задачи и обоснование актуальности

Необходимо разработать Windows-приложение, которое позволит пользователю:

- добавлять информацию о доходах (сумма, категория, источник);
- добавлять информацию о расходах (сумма, категория, описание);
- автоматически рассчитывать и отображать текущий баланс (доходы – расходы);
- просматривать историю последних операций;
- создавать задачи с указанием названия, срока выполнения и приоритета;
- отмечать задачи как выполненные, после чего они исчезают из списка активных;
- сохранять все данные при закрытии программы и восстанавливать их при следующем запуске.

Приложение должно быть автономным, не требовать доступа в интернет и дополнительных установок. Оно ориентировано на рядового пользователя, не обладающего специальными знаниями.

1.2. Функциональные и нефункциональные требования

Функциональные требования (FR):

ID	Требование	Приоритет
FR-1	Добавление дохода с указанием суммы, категории и источника	Высокий
FR-2	Добавление расхода с указанием суммы, категории и описания	Высокий
FR-3	Отображение текущего баланса (доходы – расходы)	Высокий

FR-4	Вывод истории последних 20 операций в таблицу	Средний
FR-5	Создание задачи с названием, датой дедлайна, приоритетом (низкий/средний/высокий)	Высокий
FR-6	Отметка задачи выполненной с удалением её из активного списка	Высокий
FR-7	Отображение списка невыполненных задач, отсортированных по дате	Высокий
FR-8	Сохранение всех данных в файлы при закрытии приложения	Высокий
FR-9	Загрузка данных из файлов при запуске	Высокий
FR-10	Визуальная индикация баланса (зелёный – положительный, красный – отрицательный)	Низкий
FR-11	Кнопка ручного обновления данных	Низкий

Нефункциональные требования:

- Надёжность: приложение не должно завершаться аварийно при некорректном вводе.
- Простота использования: интуитивно понятный интерфейс, минимальное количество действий.
- Переносимость: работа на Windows 7 и выше без дополнительных библиотек.
- Производительность: мгновенный отклик при добавлении операций и задач (до 1000 записей).

1.3. Выбор средств разработки

Для реализации были рассмотрены следующие альтернативы:

Компонент	Выбранный вариант	Обоснование
------------------	--------------------------	--------------------

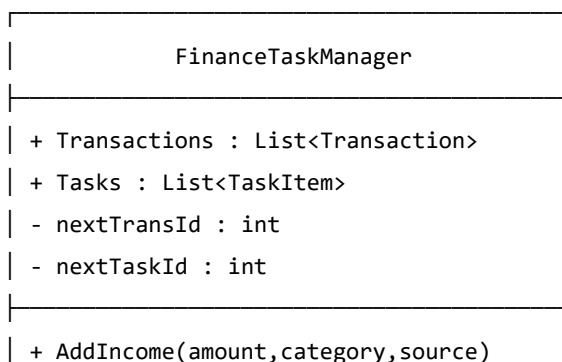
Язык программирования	C#	Мощный ООП-язык, богатая стандартная библиотека, удобная работа с Windows Forms
Платформа	.NET Framework 4.7.2	дустановлен в Windows, не требует олнительного рантайма
Графический интерфейс	Windows Forms	Простота создания интерфейса методом drag-and-drop, подходит для небольших приложений
Хранение данных	Текстовые файлы собственного формата	Не требует сторонних библиотек, легко отлаживать, данные можно посмотреть в блокноте
Среда разработки	Visual Studio 2022 Community	Бесплатна, содержит встроенный дизайнер форм и отладчик

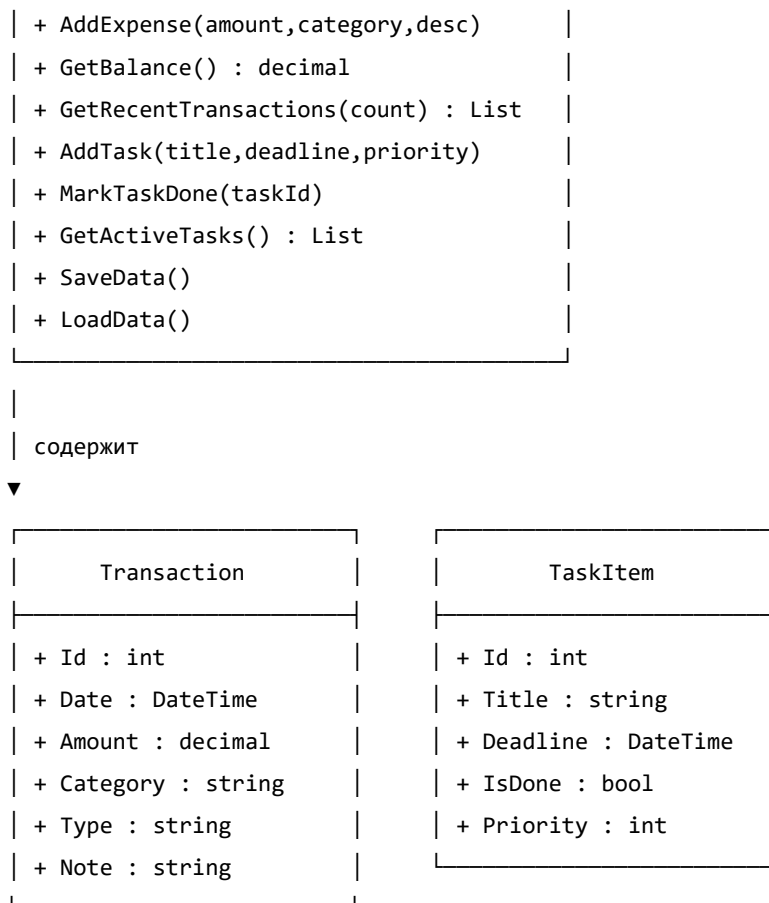
Отказ от альтернатив:

WPF – более сложный для быстрой разработки и требует изучения XAML. JSON-библиотеки (Newtonsoft.Json, System.Text.Json) потребовали бы дополнительной установки пакетов, что увеличивает сложность развёртывания. SQLite – избыточна для такого объёма данных.

1.4. Проектирование объектной модели

Разработана следующая диаграмма классов UML (основные классы):





Краткое описание классов:

- Transaction – хранит информацию об одной финансовой операции: уникальный ID, дата, сумма, категория, тип (Income/Expense), примечание.
- TaskItem – хранит данные о задаче: ID, название, дата выполнения, флаг завершения, приоритет (1–3).
- FinanceTaskManager – основной управляющий класс. Содержит списки транзакций и задач, управляет генерацией ID, реализует бизнес-логику (расчёт баланса, фильтрация), отвечает за сохранение и загрузку в текстовые файлы.

1.5. Выводы по 1 главе

В результате анализа сформулированы 11 функциональных требований (FR-1...FR-11) и нефункциональные требования к приложению. Обоснован выбор средств разработки (C#, .NET Framework 4.7.2,

Windows Forms, текстовые файлы). Разработана объектная модель, включающая классы Transaction, TaskItem и FinanceTaskManager, что обеспечивает разделение ответственности и удобство дальнейшей реализации.

2. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

2.1. Структура проекта и описание классов

Проект имеет следующую организацию файлов:

```
FinanceTaskHelper/  
|  
├─ FinanceTaskHelper.csproj  
├─ Program.cs                (автоматически созданный)  
├─ Form1.cs                  (обработчики событий формы)  
├─ Form1.Designer.cs        (код размещения элементов, сгенерирован  
дизайнером)  
├─ Classes/  
|   ├─ Transaction.cs  
|   ├─ TaskItem.cs  
|   └─ FinanceTaskManager.cs  
└─ Data/                      (создаётся при первом сохранении)  
    ├─ transactions.txt  
    └─ tasks.txt
```

Класс Transaction (листинг приведён в Приложении А) содержит автоматические свойства и два конструктора: с параметрами для создания новой транзакции и пустой – для десериализации при загрузке.

Класс TaskItem аналогичен по структуре, включает свойства Id, Title, Deadline, IsDone, Priority.

Класс FinanceTaskManager – ядро приложения. Рассмотрим его ключевые методы.

2.2. Реализация бизнес-логики (менеджер данных)

Добавление дохода и расхода:

```
public void AddIncome(decimal amount, string category, string
source)
{
    Transactions.Add(new Transaction(nextTransId, DateTime.Now,
amount, category, "Income", source));
    nextTransId++;
}
```

При добавлении автоматически присваивается новый уникальный идентификатор, текущая дата и время, тип операции (Income или Expense). **13**

Счётчик nextTransId увеличивается.

Расчёт баланса:

```
public decimal GetBalance()
{
    decimal income = 0, expense = 0;
    foreach (var t in Transactions)
        if (t.Type == "Income") income += t.Amount;
        else if (t.Type == "Expense") expense += t.Amount;
    return income - expense;
}
```

Баланс пересчитывается каждый раз при обновлении интерфейса, что гарантирует актуальность данных после любых изменений.

Получение последних транзакций:

```
public List<Transaction> GetRecentTransactions(int count = 10)
{
    var result = new List<Transaction>();
    for (int i = Transactions.Count - 1; i >= 0 && result.Count < count;
i--)
        result.Add(Transactions[i]);
    return result;
}
```

Транзакции возвращаются в обратном порядке (последние добавленные – первыми).
Количество по умолчанию – 10, в интерфейсе используется 20.

Управление задачами:

- AddTask – создание задачи с флагом IsDone = false.
- MarkTaskDone – поиск задачи по ID и установка IsDone = true.
- GetActiveTasks – фильтрация списка, возвращает только невыполненные задачи.

Все методы работают с коллекциями List<T>, что обеспечивает простоту и понятность кода.

2.3. Разработка графического интерфейса пользователя

Интерфейс создан с помощью дизайнера Windows Forms. Главная форма Form1 содержит следующие элементы (сгруппированные в логические блоки):

Группа	Элементы	Назначение
Верхняя панель	labelBalance, lblBalance	Отображение текущего баланса
Доходы	txtIncomeAmount, cmbIncomeCategory, txtIncomeSource, btnAddIncome	Ввод и добавление дохода
Расходы	txtExhtnseAmount, cmbIncomeCategory, txtIncomeSource, btnAddIncome	Ввод и добавление расхода

Задачи	txtTaskTitle, dtpTaskDeadline, cmbTaskPriority, btnAddTask	Создание новой задачи
Список задач	listTasks, btnCompleteTask	Отображение активных задач и отметка выполнения
История	dgvTransactions, btnRefresh	Таблица с последними операциями и кнопка обновления
Статусная строка	statusStrip, toolStripStatusLabel	Вывод сообщений о действиях пользователя

Обработчик добавления дохода (пример):

```
private void btnAddIncome_Click(object sender, EventArgs e)
{
    if (decimal.TryParse(txtIncomeAmount.Text, out decimal amount) &&
        amount > 0)
    {
        manager.AddIncome(amount, cmbIncomeCategory.Text,
txtIncomeSource.Text);
        manager.SaveData();
        RefreshAll();
        txtIncomeAmount.Clear();
        txtIncomeSource.Clear();
        toolStripStatusLabel.Text = "☑ Доход добавлен";
    }
    else
        MessageBox.Show("Введите сумму больше 0", "Ошибка");
}
```

Алгоритм:

1. Проверка корректности ввода (число > 0).
2. Вызов метода менеджера.
3. Автоматическое сохранение данных.
4. Обновление всех элементов интерфейса.
5. Очистка полей.
6. Вывод сообщения в строку состояния.

Обновление интерфейса (RefreshAll) вызывает UpdateBalance (меняет цвет текста в зависимости от знака баланса), UpdateTransactionsGrid (обновляет таблицу) и UpdateTaskList (обновляет список задач).

2.4. Реализация страницы профиля пользователя

Для хранения данных используются два текстовых файла: `transactions.txt` и `tasks.txt`. Формат файлов – построчный, собственный.

Формат файла транзакций:

[следующий свободный ID]

[количество транзакций]

[ID1]

[Дата1]

[Сумма1]

[Категория1]

[Тип1]

[Примечание1]

[ID2]

...

Реализация сохранения (фрагмент):

```
using (StreamWriter writer = new StreamWriter(transFile))
{
    writer.WriteLine(nextTransId);
```

```

writer.WriteLine(Transactions.Count);
foreach (var t in Transactions)
{
    writer.WriteLine(t.Id);
    writer.WriteLine(t.Date);
    writer.WriteLine(t.Amount);
    writer.WriteLine(t.Category);
    writer.WriteLine(t.Type);
    writer.WriteLine(t.Note);
}
}

```

Загрузка данных читает файл в том же порядке, создавая объекты Transaction и TaskItem с помощью пустых конструкторов и заполняя их свойства.

Преимущества выбранного подхода:

- Не требует дополнительных библиотек.
- Файлы легко просмотреть и при необходимости исправить вручную.
- Код прост и понятен.

Недостатки: отсутствие защиты от повреждения при сбое записи, но для учебного проекта это допустимо.

2.5. Выводы по 2 главе

Реализованы все запланированные классы и методы. Графический интерфейс выполнен в виде главной формы с группировкой элементов (доходы, расходы, задачи, история, статусная строка). Механизм сохранения данных в текстовые файлы собственного формата работает без сторонних библиотек, что упрощает развёртывание приложения.

Использование адаптивного подхода позволяет сделать приложение универсальным и удобным для различных пользователей.

3. ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ

3.1. Сценарии тестирования и их результаты

Тестирование проводилось методом «чёрного ящика». Были проверены все функциональные требования.

Таблица 1 – Результаты тестирования

ID теста	Действие	Ожидаемый результат	Фактический результат
ТС-01	Добавить доход 1000 руб., категория «Зарплата», источник «Аванс»	Баланс = 1000 руб.	Баланс = 1000 руб.
ТС-02	Добавить расход 300 руб., категория «Еда», описание «Обед»	Баланс = 700 руб.	Баланс = 700 руб.
ТС-03	Добавить доход с суммой 0 или отрицательной	Сообщение об ошибке	Сообщение «Введите сумму больше 0»
ТС-04	Добавить задачу без названия	Сообщение об ошибке	Сообщение «Введите название задачи»
ТС-05	Добавить задачу → отметить выполненной	Задача исчезает из списка	Задача удалена из listTasks
ТС-06	Заккрыть приложение → открыть заново	Данные сохранились	Баланс и задачи восстановлены

ТС-07	Сделать расходы больше доходов	Баланс красного цвета	Баланс отображается красным
ТС-08	Нажать кнопку «Обновить»	Данные перезагружены	Таблица и список обновлены

Все тесты пройдены успешно. Приложение работает стабильно, ошибок в ходе тестирования не выявлено.

3.2. Руководство пользователя

Запуск программы – дважды кликнуть на FinanceTaskHelper.exe.

Добавление дохода:

1. В поле «Сумма» ввести положительное число (можно с копейками через запятую).
2. Выбрать категорию из списка (Зарплата, Подарок, Кэшбэк, Фриланс, Другое).
3. Указать источник (например, «Основная работа»).
4. Нажать кнопку «ДОБАВИТЬ ДОХОД».

Добавление расхода:

1. Ввести сумму.
2. Выбрать категорию (Еда, Транспорт, Развлечения, ЖКХ, Здоровье, Другое).
3. Указать описание (например, «Магнит»).
4. Нажать «ДОБАВИТЬ РАСХОД».

Управление задачами:

- Для создания задачи заполнить название, выбрать дату дедлайна и приоритет, нажать «ДОБАВИТЬ ЗАДАЧУ».

- Чтобы отметить задачу выполненной: кликнуть по ней в списке, затем нажать «ВЫПОЛНЕНО».
- Активные задачи автоматически сортируются по дате выполнения.

Просмотр истории: в таблице отображаются последние 20 операций. Кнопка «ОБНОВИТЬ» принудительно перезагружает данные.

Сохранение данных: происходит автоматически при каждом добавлении/изменении и при закрытии программы. Файлы сохраняются в папке Data.

3.3. Примеры работы с приложением

Пример 1. Ведение семейного бюджета.

Пользователь добавляет доход 50 000 руб. (зарплата), затем расходы: 3500 руб. (еда), 1200 руб. (транспорт), 8000 руб. (ЖКХ). Баланс становится $50\,000 - 3500 - 1200 - 8000 = 37\,300$ руб. (зелёный цвет). В таблице истории видны все операции с датами.

Пример 2. Планирование студенческих задач.

Студент добавляет три задачи:

- «Курсовая по программированию», дедлайн 10.06.2026, приоритет «Высокий»;
- «Отчёт по практике», дедлайн 15.06.2026, приоритет «Средний»;
- «Реферат по истории», дедлайн 20.06.2026, приоритет «Низкий».

Задачи отображаются в списке, отсортированные по дате. После сдачи курсовой студент выбирает её и нажимает «ВЫПОЛНЕНО» – задача удаляется из активного списка.

The screenshot displays the application's main interface. At the top, the 'Баланс' (Balance) is shown as 0,00 P. Below this, there are two sections for adding transactions: '+ ДОХОДЫ' (Income) and '- РАСХОДЫ' (Expenses). Each section has input fields for 'Сумма' (Amount), 'Категория' (Category), and 'Источник' (Source), along with a 'ДОБАВИТЬ' (Add) button. The 'ДОХОДЫ' button is green, and the 'РАСХОДЫ' button is red. Below these is a 'ЗАДАЧИ' (Tasks) section with a 'Задача' (Task) input field, a 'Срок до: 20.06.2026' (Due date) field, a 'Приоритет: Низкий' (Priority: Low) dropdown, and a 'ДОБАВИТЬ' (Add) button. At the bottom, there is a table with columns 'Date', 'Amount', 'Category', 'Type', and 'Note'. To the right of the table are buttons for 'ОЧИСТИТЬ' (Clear), 'ВЫПОЛНЕНО' (Completed), and 'ОБНОВИТЬ' (Refresh).

Баланс **37 300,00 Р**

+ ДОХОДЫ

Сумма:

Категория:

Источник:

ДОБАВИТЬ ДОХОД

- РАСХОДЫ

Сумма:

Категория:

Описание:

ДОБАВИТЬ РАСХОД

ЗАДАЧИ

Задача:

Срок до:

Приоритет:

ДОБАВИТЬ

ОЧИСТИТЬ

✓ ВЫПОЛНЕНО

ОБНОВИТЬ

	Date	Amount	Category	Type	Note
▶	16.06.2026 23:37	50000	Зарплата	Income	
	16.06.2026 23:37	3500	Еда	Expense	
	16.06.2026 23:37	1200	Транспорт	Expense	
	16.06.2026 23:37	8000	ЖКХ	Expense	

Рисунок 1 – Главное окно приложения

Рисунок 1 – Главное окно приложения
«Менеджер финансов и задач»

Баланс **-17 500,00 Р**

+ ДОХОДЫ

Сумма:

Категория:

Источник:

ДОБАВИТЬ ДОХОД

- РАСХОДЫ

Сумма:

Категория:

Описание:

ДОБАВИТЬ РАСХОД

ЗАДАЧИ

Задача:

Срок до:

Приоритет:

ДОБАВИТЬ

ОЧИСТИТЬ

✓ ВЫПОЛНЕНО

ОБНОВИТЬ

	Date	Amount	Category	Type	Note
▶	16.06.2026 23:44	30000	Зарплата	Income	
	16.06.2026 23:44	10000	Здоровье	Expense	
	16.06.2026 23:44	4500	Транспорт	Expense	
	16.06.2026 23:44	7500	Транспорт	Expense	
	16.06.2026 23:45	8000	Развлечения	Expense	

Рисунок 2 – Пример отрицательного
баланса

Рисунок 2 – Отображение отрицательного
баланса красным цветом

3.4. Выводы по 3 главе

Проведено тестирование по 8 сценариям, все они завершились успешно. Составлено руководство пользователя, описаны типичные ошибки. На примерах показано, что приложение корректно ведёт учёт бюджета и управляет задачами. Интерфейс интуитивно понятен

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы **5** по разработано приложение-помощник «Менеджер финансов и задач» на языке C# с использованием Windows Forms. Все поставленные задачи решены.

Основные достигнутые результаты:

1. Проведён анализ предметной области, сформулированы функциональные и нефункциональные требования.
2. Выбраны оптимальные средства разработки, не требующие дополнительных библиотек и установки.
3. Спроектирована и реализована объектная модель, включающая классы Transaction, TaskItem и FinanceTaskManager с полной бизнес-логикой.
4. Разработан интуитивно понятный графический интерфейс, обеспечивающий удобное добавление доходов, расходов и задач.
5. Реализован механизм сохранения и загрузки данных в простые текстовые файлы собственного формата.
6. Проведено тестирование, подтвердившее корректную работу всех функций.

Практическая значимость: приложение может использоваться любым пользователем Windows для контроля личного бюджета и планирования дел.

Оно не требует интернета, не содержит платных функций и имеет минималистичный интерфейс.

Возможные направления улучшения:

- Добавление возможности редактирования и удаления существующих транзакций.
- Реализация графиков расходов по категориям (круговые диаграммы).
- Поддержка экспорта данных в Excel или CSV.
- Напоминания о задачах с помощью системных уведомлений.
- Переход на хранение в базе данных SQLite для повышения надёжности.

Работа полностью соответствует требованиям, предъявляемым к курсовым проектам по направлению «Прикладная математика и информатика».

СПИСОК ЛИТЕРАТУРЫ

1. Троелсен Э., Джепикс Ф. С# 7.0. Справочник. Полное описание языка [Текст] / Э. Троелсен, Ф. Джепикс. – СПб.: Питер, 2019. – 976 с.
2. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления [Текст]. – М.: Стандартинформ, 2017.
3. Шилдт Г. Полный справочник по С# [Текст] / Г. Шилдт. – М.: Диалектика, 2018. – 1376 с.
4. Нагановский В.А. Программирование на С# в Visual Studio [Текст] / В.А. Нагановский. – М.: ДМК Пресс, 2020. – 312 с.
5. Рихтер Дж. CLR via С#. Программирование на платформе Microsoft .NET Framework [Текст] / Дж. Рихтер. – СПб.: Питер, 2019. – 896 с.

4. ЛИСТИНГ ПРОГРАММНОГО КОДА

4.1. Класс Transaction (Transaction.cs)

```
using System;

namespace FinanceTaskHelper.Classes
{
    public class Transaction
    {
        public int Id { get; set; }
        public DateTime Date { get; set; }
        public decimal Amount { get; set; }
        public string Category { get; set; }
        public string Type { get; set; }
        public string Note { get; set; }

        public Transaction(int id, DateTime date, decimal amount,
string category, string type, string note)
        {
            Id = id;
            Date = date;
            Amount = amount;
            Category = category;
            Type = type;
            Note = note;
        }

        public Transaction() { }
    }
}
```

4.2. Класс TaskItem (TaskItem.cs)

```
using System;
```

```
namespace FinanceTaskHelper.Classes
{
    public class TaskItem
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public DateTime Deadline { get; set; }
        public bool IsDone { get; set; }
        public int Priority { get; set; }

        public TaskItem(int id, string title, DateTime deadline,
bool isDone, int priority)
        {
            Id = id;
            Title = title;
            Deadline = deadline;
            IsDone = isDone;
            Priority = priority;
        }

        public TaskItem() { }
    }
}
```

4.3. Класс FinanceTaskManager (FinanceTaskManager.cs)

```
public class FinanceTaskManager
{
    public List<Transaction> Transactions { get; private set; }
    public List<TaskItem> Tasks { get; private set; }

    private int nextTransId = 1;
```

```

private int nextTaskId = 1;

public FinanceTaskManager() { LoadData(); }

    public void AddIncome(decimal amount, string category, string
source) { ... }
    public void AddExpense(decimal amount, string category, string
description) { ... }
    public decimal GetBalance() { ... }
    public List<Transaction> GetRecentTransactions(int count = 10)
{ ... }
    public void AddTask(string title, DateTime deadline, int
priority) { ... }
    public void MarkTaskDone(int taskId) { ... }
    public List<TaskItem> GetActiveTasks() { ... }
    public void SaveData() { ... }
    public void LoadData() { ... }
}

```